UNIVERSIDAD POLITÉCNICA DE ATLAUTLA
Fundamentos de Inteligencia Artificial · Grupo 171



ACTIVIDAD PARCIAL 2 · SECCIÓN 5

A* vs Dijkstra

Rutas en una red de transporte regional: métricas, dashboard y análisis de admisibilidad de la heurística.

2.5 Búsqueda con heurísticas (A* / Dijkstra) · Proyecto en equipo

INTEGRANTES DEL EQUIPO

- **Bernal Najera Alexandra**
- **Marin Flor Julissa**
- **Rosas Toriz Erandy Naomy**
- **Suarez Melonie Magdalena**

DOCENTE
Mtro. Javier García Lara

GRUPO
171 · Cuatrimestre 7

ASIGNATURA
Fundamentos de Inteligencia Artificial

PERIODO
12 al 26 de octubre de 2026

ENTREGA
25 de octubre de 2026 · Equipo

REPOSITORIO

01 Planteamiento del problema

Los sistemas de navegación calculan rutas sobre una red de carreteras: los cruces o ciudades son nodos y los tramos son aristas con un costo (distancia o tiempo). Dos algoritmos clásicos resuelven el problema de la **ruta de menor costo**. **Dijkstra** expande los nodos en orden de costo acumulado g(n), sin saber dónde está el destino. **A*** agrega una **heurística** h(n) que estima lo que falta y expande primero los nodos con menor f(n) = g(n) + h(n) (Hart et al., 1968).

> **PREGUNTA GUÍA**
> ¿Cuánto trabajo ahorra A* frente a Dijkstra, bajo qué condiciones sigue encontrando la ruta óptima y qué pasa cuando la heurística no es admisible?

◆ Caso de estudio

Se modeló una red carretera regional del centro de México con **21 ciudades y 33 carreteras** (coordenadas y conexiones aproximadas). El viaje es de **Atlautla a Querétaro**. Para ver cómo escalan los algoritmos también se generaron redes aleatorias de 100 a 2,500 nodos.

◆ Objetivos

- Implementar Dijkstra y A* y compararlos con métricas: costo de la ruta, nodos expandidos y tiempo.
- Construir un dashboard en matplotlib con los resultados.
- Comprobar si la heurística es admisible y consistente, y medir el efecto de una heurística no admisible.

02 Formulación del problema

Elemento	Formulación
Estado	Ciudad en la que se encuentra el viajero.
Estado inicial / meta	Atlautla / Querétaro.
Acciones	Viajar a una ciudad conectada por carretera.
Costo de paso c(u, v)	Distancia por carretera estimada en km = distancia en línea recta × 1.25.
g(n)	Costo acumulado real desde el origen hasta n.
h(n) (heurística)	Distancia en línea recta (fórmula de Haversine) de n al destino.
f(n)	g(n) + h(n): prioridad de A*. Dijkstra usa solo g(n).

◆ Admisibilidad y consistencia

- Una heurística es **admisible** si nunca sobrestima el costo real: h(n) ≤ h*(n) para todo n. Con una heurística admisible, A* devuelve una ruta óptima.
- Es **consistente** si h(u) ≤ c(u, v) + h(v) en cada arista (desigualdad triangular). Toda heurística consistente es admisible, y A* no necesita reabrir nodos.
- La línea recta es la distancia más corta posible entre dos puntos, y una carretera nunca es más corta. Como el factor de sinuosidad es 1.25 ≥ 1, h es admisible y consistente por construcción; el código lo

verifica.

03 Datos y procedimiento

◆ Datos de entrada

Ciudad	Lat.	Lon.	Ciudad	Lat.	Lon.
CDMX	19.43	-99.13	Tulancingo	20.08	-98.37
Texcoco	19.51	-98.88	Toluca	19.28	-99.66
Chalco	19.26	-98.90	Querétaro	20.59	-100.39
Amecameca	19.13	-98.77	San Juan del Río	20.39	-99.99
Atlautla	19.02	-98.77	Tula	20.05	-99.34
Cuautla	18.81	-98.95	Tepeji	19.90	-99.35
Cuernavaca	18.92	-99.23	Cholula	19.06	-98.30
Puebla	19.04	-98.21	Atlixco	18.91	-98.43
Tlaxcala	19.31	-98.24	Izúcar	18.60	-98.47
Apizaco	19.41	-98.14	Tehuacán	18.46	-97.39
Pachuca	20.10	-98.76			

Tabla 1. Ciudades y coordenadas aproximadas (grados).

Carretera	km	Carretera	km	Carretera	km
CDMX–Texcoco	34.6	Texcoco–Tlaxcala	88.4	Atlixco–Cholula	27.0
CDMX–Chalco	38.3	Texcoco–Chalco	34.8	Cholula–Puebla	12.1
CDMX–Toluca	72.6	Chalco–Amecameca	24.9	Puebla–Tlaxcala	37.7
CDMX–Cuernavaca	72.1	Amecameca–Atlautla	15.3	Tlaxcala–Apizaco	19.1
CDMX–Pachuca	105.0	Atlautla–Cuautla	37.6	Apizaco–Puebla	52.2
CDMX–Tepeji	71.4	Chalco–Puebla	95.6	Apizaco–Tulancingo	97.9
Tepeji–Tula	20.9	Amecameca–Puebla	74.6	Pachuca–Tulancingo	51.0
Tula–San Juan del Río	97.1	Cuautla–Cuernavaca	39.9	Puebla–Tehuacán	134.7
San Juan del Río–Querétaro	59.0	Cuautla–Izúcar	69.6	Izúcar–Tehuacán	143.7
Tula–Pachuca	76.0	Izúcar–Atlixco	43.4	Toluca–Cuernavaca	75.5
Texcoco–Pachuca	83.5	Atlixco–Puebla	34.1	Toluca–Tepeji	95.3

Tabla 2. Carreteras y su costo estimado en km. Son datos ilustrativos, no mediciones reales.

Redes aleatorias. Para medir el escalamiento se generaron redes de 100 a 2,500 nodos en un cuadrado de 100 × 100 km. Cada nodo se conecta con sus 5 vecinos más cercanos y el costo de cada arista es su distancia euclidiana por un factor aleatorio entre 1.0 y 1.5 (así la heurística sigue siendo admisible). Cada tamaño se promedia sobre 30 consultas de origen y destino aleatorios.

◆ Procedimiento

- Construir la red como grafo ponderado con NetworkX.
- Ejecutar Dijkstra y A* con la misma estructura (cola de prioridad) y registrar costo, nodos expandidos y tiempo.
- Verificar el costo con `nx.dijkstra_path_length` y que ambos algoritmos devuelvan el mismo costo.
- Comparar h(n) con el costo real h*(n) (Dijkstra desde el destino) para comprobar admisibilidad y consistencia.
- Repetir con la heurística multiplicada por 1.5, 2 y 3 para observar qué pasa cuando deja de ser admisible.

```
A*(grafo, origen, destino, h):
    abiertos <- cola de prioridad [(h(origen), origen)] ; g[origen] <- 0
    mientras abiertos no este vacia:
        u <- sacar el nodo con menor f = g + h
        si u es destino: devolver camino y g[u]
        para cada vecino v de u con costo c:
            si g[u] + c < g[v]:
                g[v] <- g[u] + c ; padre[v] <- u
                insertar (g[v] + h(v), v) en abiertos
Dijkstra es el mismo algoritmo con h(n) = 0.
```

Pseudocódigo de A (el código completo está en el Anexo A).*

◆ Instructivo para replicar los resultados

Paso	Acción
1	Instalar Python 3.10 o superior.
2	Instalar las librerías: `pip install networkx numpy matplotlib`
3	Descargar la carpeta `S5_astar_dijkstra` del repositorio.
4	Ejecutar: `python s5_astar_dijkstra.py`
5	Ver los dashboards en la carpeta `figuras/` y los datos en `resultados_s5.json`.

04 Dashboard y resultados

◆ 4.1 Red regional: Atlautla → Querétaro

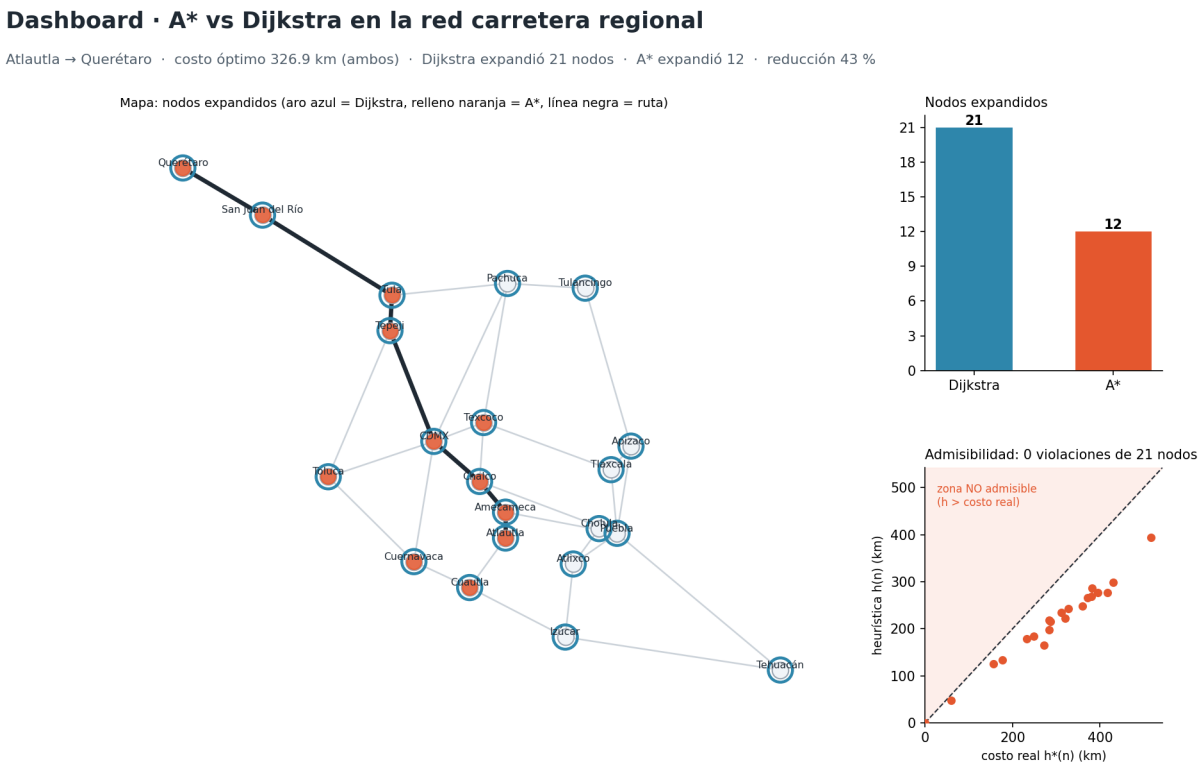


Figura 1. Dashboard de la red regional: mapa con los nodos expandidos por cada algoritmo, nodos expandidos y prueba de admisibilidad.

Métrica	Dijkstra	A*
Costo de la ruta (km)	326.9	326.9
Nodos expandidos	21	12 (−43 %)
Tiempo (ms)	0.086	0.058
Ruta	Atlautla → Amecameca → Chalco → CDMX → Tepeji → Tula → San Juan del Río → Querétaro	la misma

Tabla 3. Comparación en la red regional. Los dos algoritmos encuentran la misma ruta óptima; A* revisa menos ciudades. El tiempo en una red de 21 nodos es de microsegundos y varía entre ejecuciones.

◆ 4.2 Escalamiento

Dashboard · Escalamiento de A* y Dijkstra y efecto de una heurística no admisible

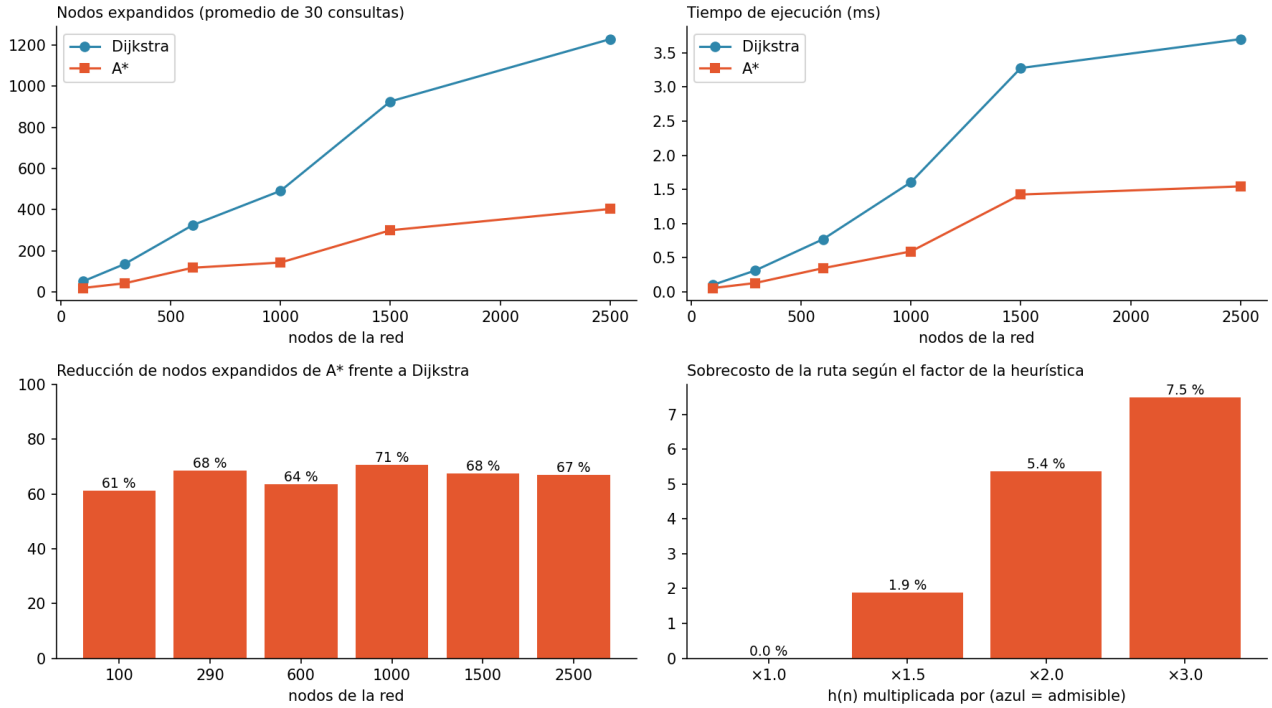


Figura 2. Dashboard de escalamiento: nodos expandidos, tiempo, reducción y efecto de una heurística no admisible.

Nodos de la red	Dijkstra: expandidos	A*: expandidos	Dijkstra (ms)	A* (ms)	Reducción
100	52	20	0.10	0.05	61 %
290	136	43	0.31	0.13	68 %
600	325	118	0.77	0.34	64 %
1,000	492	144	1.60	0.59	71 %
1,500	926	300	3.28	1.42	68 %
2,500	1,228	404	3.70	1.54	67 %

Tabla 4. Promedio de 30 consultas por tamaño. En todas las consultas ambos algoritmos devolvieron el mismo costo (el código lo comprueba con un assert).

05 Análisis de admisibilidad

En la red regional se comparó h(n) con el costo real h*(n) para los 21 nodos y se revisaron las 33 aristas: hubo **0 violaciones de admisibilidad** y **0 violaciones de consistencia**. La razón máxima h/h* fue 0.80, es decir, la heurística nunca supera el costo real y en el peor caso lo subestima en un 20 %.

◆ ¿Qué pasa si la heurística no es admisible?

Se multiplicó h(n) por un factor mayor que 1 en una red de 600 nodos con 60 consultas. Cuanto más se infla la heurística, más rápido va A* hacia el destino, pero deja de garantizar la ruta óptima.

Factor de h	¿Admisible?	Nodos expandidos	Rutas no óptimas	Sobrecosto medio
× 1.0	Sí	102	0 de 60	0.00 %
× 1.5	No	29	45 de 60	1.89 %
× 2.0	No	19	55 de 60	5.38 %
× 3.0	No	17	56 de 60	7.48 %

Tabla 5. Efecto de inflar la heurística (sobrecosto = aumento del costo frente a la ruta óptima).

LECTURA DEL RESULTADO
Con h × 1 A* expandió 102 nodos y siempre dio la ruta óptima. Con h × 3 expandió solo 17 nodos, pero la ruta fue en promedio 7.5 % más larga. Esta es la idea de A* ponderado: se acepta una ruta algo peor a cambio de menos cálculo.

06 Conclusiones reflexivas

- A* con una heurística admisible obtiene la misma ruta óptima que Dijkstra y revisa muchos menos nodos: 43 % menos en la red regional y entre 61 % y 71 % en las redes aleatorias.
- La ventaja de A* crece con el tamaño de la red, porque Dijkstra se expande en todas direcciones y A* se dirige hacia el destino.
- La calidad de la heurística lo decide todo. Una heurística admisible garantiza optimalidad; una inflada acelera la búsqueda pero puede dar rutas peores, como lo muestra la Tabla 5.
- Limitaciones: la red regional es pequeña, sus costos son estimados (no tiempos reales con tráfico) y los tiempos de ejecución son muy pequeños y varían con la computadora, por lo que la comparación más confiable es la de nodos expandidos. Con datos reales (por ejemplo, de OpenStreetMap) habría que usar tiempo de viaje y una heurística basada en la velocidad máxima.

07 Declaración de uso de inteligencia artificial

Herramienta	Claude (Anthropic, 2026), modelo Claude Sonnet 5.5, con ejecución de código Python.
Prompt (resumen)	Se entregó la hoja de la Actividad Parcial 2 y se pidió el código y el PDF de la sección 5 (2.5 Búsqueda con heurísticas, A* y Dijkstra, proyecto en equipo).
Uso	Diseño del caso y de los experimentos, escritura del código (Dijkstra, A*, verificación de la heurística, dashboards), borrador del texto y maquetación del PDF (reportlab).
Verificación	El código se ejecutó completo; el costo de la ruta se contrastó con NetworkX y se comprobó que A* y Dijkstra dieran el mismo costo en todas las consultas. Las coordenadas y distancias son aproximadas. Las referencias deben cotejarse con las fuentes originales antes de entregar.

R Referencias (APA 7.ª ed.)

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269-271. https://doi.org/10.1007/BF01386390

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11-15).

Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics, 4*(2), 100-107. https://doi.org/10.1109/TSSC.1968.300136

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un enfoque moderno* (4.ª ed.). Pearson.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky and Telescope, 68*(2), 159.

A Anexo A. Código Python

Archivo: s5_astar_dijkstra.py

```
"""S5 · A* y Dijkstra en una red de transporte (proyecto en equipo + dashboard)

Compara A* y Dijkstra sobre:
  1) Una red carretera regional (21 ciudades del centro de México, coordenadas aproximadas).
  2) Redes aleatorias de 100 a 2,500 nodos para ver cómo escala cada algoritmo.
Además comprueba si la heurística (distancia en línea recta) es admisible y consistente,
y muestra qué pasa cuando se usa una heurística NO admisible.

Equipo: Bernal Najera Alexandra · Marin Flor Julissa · Rosas Toriz Erandy Naomy · Suarez Melonie Magdalena
Ejecutar: python s5_astar_dijkstra.py
Requiere: pip install networkx numpy matplotlib
Genera:   figuras/dashboard_red_regional.png y figuras/dashboard_escalamiento.png
"""
import heapq
import json
import math
import os
import random
import time

import matplotlib
import matplotlib.ticker
matplotlib.use("Agg")  # genera PNG sin abrir ventana
import matplotlib.pyplot as plt
import networkx as nx
import numpy as np

AQUI = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(AQUI, "figuras")
os.makedirs(FIG, exist_ok=True)

# ============================================================================
# 1. Datos: red carretera regional (coordenadas y conexiones aproximadas)
# ============================================================================
CIUDADES = {  # nombre: (latitud, longitud)
    "CDMX": (19.43, -99.13), "Texcoco": (19.51, -98.88), "Chalco": (19.26, -98.90),
    "Amecameca": (19.13, -98.77), "Atlautla": (19.02, -98.77), "Cuautla": (18.81, -98.95),
    "Cuernavaca": (18.92, -99.23), "Puebla": (19.04, -98.21), "Tlaxcala": (19.31, -98.24),
    "Apizaco": (19.41, -98.14), "Pachuca": (20.10, -98.76), "Tulancingo": (20.08, -98.37),
    "Toluca": (19.28, -99.66), "Querétaro": (20.59, -100.39), "San Juan del Río": (20.39, -99.99),
    "Tula": (20.05, -99.34), "Tepeji": (19.90, -99.35), "Cholula": (19.06, -98.30),
    "Atlixco": (18.91, -98.43), "Izúcar": (18.60, -98.47), "Tehuacán": (18.46, -97.39),
}
CARRETERAS = [
    ("CDMX", "Texcoco"), ("CDMX", "Chalco"), ("CDMX", "Toluca"), ("CDMX", "Cuernavaca"),
    ("CDMX", "Pachuca"), ("CDMX", "Tepeji"), ("Tepeji", "Tula"), ("Tula", "San Juan del Río"),
    ("San Juan del Río", "Querétaro"), ("Tula", "Pachuca"), ("Texcoco", "Pachuca"),
    ("Texcoco", "Tlaxcala"), ("Texcoco", "Chalco"), ("Chalco", "Amecameca"), ("Amecameca", "Atlautla"),
    ("Atlautla", "Cuautla"), ("Chalco", "Puebla"), ("Amecameca", "Puebla"), ("Cuautla", "Cuernavaca"),
    ("Cuautla", "Izúcar"), ("Izúcar", "Atlixco"), ("Atlixco", "Puebla"), ("Atlixco", "Cholula"),
    ("Cholula", "Puebla"), ("Puebla", "Tlaxcala"), ("Tlaxcala", "Apizaco"), ("Apizaco", "Puebla"),
    ("Apizaco", "Tulancingo"), ("Pachuca", "Tulancingo"), ("Puebla", "Tehuacán"), ("Izúcar", "Tehuacán"),
    ("Toluca", "Cuernavaca"), ("Toluca", "Tepeji"),
]
SINUOSIDAD = 1.25   # la carretera mide ~25 % más que la línea recta (nunca menos)


def haversine(a, b):
    """Distancia en km sobre la esfera entre dos puntos (lat, lon)."""
    la1, lo1, la2, lo2 = map(math.radians, (*a, *b))
    h = math.sin((la2 - la1) / 2) ** 2 + math.cos(la1) * math.cos(la2) * math.sin((lo2 - lo1) / 2) ** 2
    return 2 * 6371.0 * math.asin(math.sqrt(h))


def euclidiana(a, b):
    return math.hypot(a[0] - b[0], a[1] - b[1])


def red_regional():
    G = nx.Graph(dist=haversine)
    for n, p in CIUDADES.items():
```

```
        G.add_node(n, pos=p)
    for u, v in CARRETERAS:
        G.add_edge(u, v, weight=round(haversine(CIUDADES[u], CIUDADES[v]) * SINUOSIDAD, 1))
    return G


def red_aleatoria(n, k=5, semilla=0):
    """Red de n nodos en un cuadrado de 100x100 km; cada nodo se une a sus k vecinos más cercanos.
    El peso de cada arista es la distancia euclidiana por un factor de sinuosidad >= 1."""
    rng = np.random.default_rng(semilla)
    pts = rng.uniform(0, 100, size=(n, 2))
    G = nx.Graph(dist=euclidiana)
    for i, p in enumerate(pts):
        G.add_node(i, pos=(p[0], p[1]))
    d = np.linalg.norm(pts[:, None, :] - pts[None, :, :], axis=2)
    for i in range(n):
        for j in np.argsort(d[i])[1:k + 1]:
            G.add_edge(i, int(j), weight=float(d[i, j] * rng.uniform(1.0, 1.5)))
    mayor = max(nx.connected_components(G), key=len)
    return G.subgraph(mayor).copy()


# ======================================================================
# 2. Algoritmos: Dijkstra y A* (misma estructura, solo cambia la prioridad)
# ======================================================================
def buscar(G, origen, destino, h=None):
    """Búsqueda de costo uniforme (Dijkstra) si h es None; A* si se da una heurística h(nodo).
    Prioridad = g + h. Devuelve camino, costo, nodos expandidos, tiempo y orden de expansión."""
    t0 = time.perf_counter()
    g = {origen: 0.0}
    padre = {}
    abiertos = [((h(origen) if h else 0.0), 0.0, origen)]
    cerrados = set()
    orden = []
    while abiertos:
        _, gu, u = heapq.heappop(abiertos)
        if u in cerrados:
            continue                    # entrada vieja de la cola
        cerrados.add(u)
        orden.append(u)
        if u == destino:
            break
        for v, datos in G[u].items():
            nuevo = gu + datos["weight"]
            if nuevo < g.get(v, math.inf):
                g[v] = nuevo
                padre[v] = u
                heapq.heappush(abiertos, (nuevo + (h(v) if h else 0.0), nuevo, v))
    camino = [destino]
    while camino[-1] != origen:
        camino.append(padre[camino[-1]])
    return {"camino": camino[::-1], "costo": g[destino], "expandidos": len(orden),
            "tiempo_ms": (time.perf_counter() - t0) * 1000, "orden": orden}


def dijkstra(G, o, d):
    return buscar(G, o, d)


def astar(G, o, d, escala=1.0):
    """A* con h(n) = escala x distancia en línea recta al destino. escala = 1 es admisible."""
    pos, f = nx.get_node_attributes(G, "pos"), G.graph["dist"]
    return buscar(G, o, d, lambda n: escala * f(pos[n], pos[d]))


# ======================================================================
# 3. Admisibilidad y consistencia de la heurística
# ======================================================================
def verificar_heuristica(G, destino, escala=1.0):
    """Compara h(n) con el costo real h*(n) (Dijkstra desde el destino) y revisa la consistencia."""
    pos, f = nx.get_node_attributes(G, "pos"), G.graph["dist"]
    h = {n: escala * f(pos[n], pos[destino]) for n in G}
    real = nx.single_source_dijkstra_path_length(G, destino)
    viol_adm = [n for n in G if h[n] > real[n] + 1e-9]
    viol_con = [(u, v) for u, v, d in G.edges(data=True)
                if h[u] > d["weight"] + h[v] + 1e-9 or h[v] > d["weight"] + h[u] + 1e-9]
    ratio = max(h[n] / real[n] for n in G if real[n] > 0)
    return {"h": h, "real": real, "viol_adm": len(viol_adm), "viol_con": len(viol_con),
            "max_ratio": ratio, "nodos": G.number_of_nodes(), "aristas": G.number_of_edges()}
```

```
151
152
153 # ======================================================================
154 # 4. Experimentos
155 # ======================================================================
156 def caso_regional(origen="Atlautla", destino="Querétaro"):
157     G = red_regional()
158     return G, dijkstra(G, origen, destino), astar(G, origen, destino), verificar_heuristica(G, destino)
159
160
161 def escalamiento(tamanos=(100, 300, 600, 1000, 1500, 2500), pares=30):
162     """Promedia nodos expandidos y tiempo sobre 'pares' consultas aleatorias por tamaño."""
163     filas = []
164     for n in tamanos:
165         G = red_aleatoria(n, semilla=n)
166         nodos = list(G.nodes)
167         rnd = random.Random(n)
168         d_exp, a_exp, d_t, a_t = [], [], [], []
169         for _ in range(pares):
170             o, d = rnd.sample(nodos, 2)
171             rd, ra = dijkstra(G, o, d), astar(G, o, d)
172             assert abs(rd["costo"] - ra["costo"]) < 1e-6      # A* admisible = mismo costo óptimo
173             d_exp.append(rd["expandidos"]); a_exp.append(ra["expandidos"])
174             d_t.append(rd["tiempo_ms"]); a_t.append(ra["tiempo_ms"])
175         filas.append({"n": G.number_of_nodes(), "dij_exp": np.mean(d_exp), "ast_exp": np.mean(a_exp),
176                       "dij_ms": np.mean(d_t), "ast_ms": np.mean(a_t),
177                       "reduccion": 100 * (1 - np.mean(a_exp) / np.mean(d_exp))})
178     return filas
179
180
181 def heuristica_no_admisible(escalas=(1.0, 1.5, 2.0, 3.0), n=600, pares=60):
182     """Multiplica h por un factor > 1 (la vuelve no admisible) y mide sobrecosto y nodos expandidos."""
183     G = red_aleatoria(n, semilla=7)
184     nodos = list(G.nodes)
185     filas = []
186     for e in escalas:
187         rnd = random.Random(11)
188         sobre, exp, malas = [], [], 0
189         for _ in range(pares):
190             o, d = rnd.sample(nodos, 2)
191             ra, opt = astar(G, o, d, escala=e), dijkstra(G, o, d)
192             extra = 100 * (ra["costo"] / opt["costo"] - 1)
193             sobre.append(extra); exp.append(ra["expandidos"]); malas += extra > 1e-6
194         filas.append({"escala": e, "sobrecosto": float(np.mean(sobre)), "expandidos": float(np.mean(exp)),
195                       "no_optimas": malas, "pares": pares, "admisible": verificar_heuristica(G, nodos[0], e)["viol_adm"] == 0})
196     return filas
197
198
199 # ======================================================================
200 # 5. Dashboard (matplotlib)
201 # ======================================================================
202 CORAL, GRAFITO, AZUL, GRIS = "#E4572E", "#1F2933", "#2E86AB", "#9AA5B1"
203
204
205 def dashboard_regional(G, rd, ra, ver, origen, destino):
206     fig = plt.figure(figsize=(14, 8.2))
207     gs = fig.add_gridspec(2, 3, width_ratios=[1.35, 1.35, 1], hspace=0.38, wspace=0.28, top=0.84, bottom=0.07)
208     fig.text(0.04, 0.95, "Dashboard · A* vs Dijkstra en la red carretera regional", fontsize=17, fontweight="bold", color=GRAFITO)
209     red = 100 * (1 - ra["expandidos"] / rd["expandidos"])
210     fig.text(0.04, 0.905, f"{origen} → {destino}  ·  costo óptimo {ra['costo']:.1f} km (ambos)  ·  Dijkstra expandió "
211              f"{rd['expandidos']} nodos  ·  A* expandió {ra['expandidos']}  ·  reducción {red:.0f} %", fontsize=10.5, color="#52606D")
212     pos = {n: (p[1], p[0]) for n, p in nx.get_node_attributes(G, "pos").items()}   # x = lon, y = lat
213     ax = fig.add_subplot(gs[:, :2])
214     nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#CBD2D9", width=1.2)
215     nx.draw_networkx_nodes(G, pos, ax=ax, node_color="#F0F4F8", edgecolors=GRIS, node_size=150)
216     nx.draw_networkx_nodes(G, pos, ax=ax, nodelist=rd["orden"], node_color="none", edgecolors=AZUL, linewidths=2.2, node_size=330)
217     nx.draw_networkx_nodes(G, pos, ax=ax, nodelist=ra["orden"], node_color=CORAL, alpha=0.85, node_size=120)
218     ruta = list(zip(ra["camino"][:-1], ra["camino"][1:]))
219     nx.draw_networkx_edges(G, pos, ax=ax, edgelist=ruta, edge_color=GRAFITO, width=3.2)
220     nx.draw_networkx_labels(G, pos, ax=ax, font_size=7.5, verticalalignment="bottom",
221                             labels={n: n for n in G}, font_color=GRAFITO)
222     ax.set_title("Mapa: nodos expandidos (aro azul = Dijkstra, relleno naranja = A*, línea negra = ruta)",
223                  fontsize=9.5, loc="left")
224     ax.axis("off")
225     ax2 = fig.add_subplot(gs[0, 2])
226     b = ax2.bar(["Dijkstra", "A*"], [rd["expandidos"], ra["expandidos"]], color=[AZUL, CORAL], width=0.55)
227     ax2.bar_label(b, fontsize=10, fontweight="bold")
228     ax2.set_title("Nodos expandidos", fontsize=10, loc="left"); ax2.spines[["top", "right"]].set_visible(False)
229     ax2.yaxis.set_major_locator(matplotlib.ticker.MaxNLocator(integer=True))
```

```
    ax3 = fig.add_subplot(gs[1, 2])
    h, real = [ver["h"][n] for n in G], [ver["real"][n] for n in G]
    ax3.scatter(real, h, color=CORAL, s=28, zorder=3)
    lim = max(real) * 1.05
    ax3.plot([0, lim], [0, lim], "--", color=GRAFITO, lw=1)
    ax3.fill_between([0, lim], [0, lim], lim, color="#FDE8E2", alpha=0.7, zorder=0)
    ax3.text(lim * 0.05, lim * 0.85, "zona NO admisible\n(h > costo real)", fontsize=8, color=CORAL)
    ax3.set_xlim(0, lim); ax3.set_ylim(0, lim)
    ax3.set_xlabel("costo real h*(n) (km)", fontsize=9); ax3.set_ylabel("heurística h(n) (km)", fontsize=9)
    ax3.set_title(f"Admisibilidad: {ver['viol_adm']} violaciones de {ver['nodos']} nodos", fontsize=10, loc="left")
    ax3.spines[["top", "right"]].set_visible(False)
    fig.savefig(os.path.join(FIG, "dashboard_red_regional.png"), dpi=150)
    plt.close(fig)


def dashboard_escalamiento(esc, noadm):
    fig, axs = plt.subplots(2, 2, figsize=(12, 7.6))
    fig.suptitle("Dashboard · Escalamiento de A* y Dijkstra y efecto de una heurística no admisible",
                 fontsize=14, fontweight="bold", color=GRAFITO, x=0.04, ha="left")
    n = [f["n"] for f in esc]
    a = axs[0, 0]; a.plot(n, [f["dij_exp"] for f in esc], "o-", color=AZUL, label="Dijkstra"); a.plot(n, [f["ast_exp"] for f in esc], "s
    a.set_title("Nodos expandidos (promedio de 30 consultas)", fontsize=10, loc="left"); a.set_xlabel("nodos de la red"); a.legend()
    a = axs[0, 1]; a.plot(n, [f["dij_ms"] for f in esc], "o-", color=AZUL, label="Dijkstra"); a.plot(n, [f["ast_ms"] for f in esc], "s-"
    a.set_title("Tiempo de ejecución (ms)", fontsize=10, loc="left"); a.set_xlabel("nodos de la red"); a.legend()
    a = axs[1, 0]; b = a.bar([str(x) for x in n], [f["reduccion"] for f in esc], color=CORAL); a.bar_label(b, fmt="%.0f %%", fontsize=9)
    a.set_title("Reducción de nodos expandidos de A* frente a Dijkstra", fontsize=10, loc="left"); a.set_xlabel("nodos de la red"); a.se
    a = axs[1, 1]; x = [f"×{f['escala']}" for f in noadm]
    b = a.bar(x, [f["sobrecosto"] for f in noadm], color=[AZUL if f["admisible"] else CORAL for f in noadm])
    a.bar_label(b, fmt="%.1f %%", fontsize=9)
    a.set_title("Sobrecosto de la ruta según el factor de la heurística", fontsize=10, loc="left"); a.set_xlabel("h(n) multiplicada por
    for a in axs.flat:
        a.spines[["top", "right"]].set_visible(False)
    fig.tight_layout(rect=[0, 0, 1, 0.94])
    fig.savefig(os.path.join(FIG, "dashboard_escalamiento.png"), dpi=150)
    plt.close(fig)


def main():
    origen, destino = "Atlautla", "Querétaro"
    G, rd, ra, ver = caso_regional(origen, destino)
    print(f"=== Red regional: {G.number_of_nodes()} ciudades, {G.number_of_edges()} carreteras ===")
    print(f"{'':10}{'Costo (km)':>12}{'Expandidos':>12}{'Tiempo (ms)':>13}")
    for nombre, r in (("Dijkstra", rd), ("A*", ra)):
        print(f"{nombre:10}{r['costo']:>12.1f}{r['expandidos']:>12}{r['tiempo_ms']:>13.3f}")
    print("Ruta:", " -> ".join(ra["camino"]))
    print(f"Heurística haversine: violaciones de admisibilidad = {ver['viol_adm']}, de consistencia = {ver['viol_con']}, "
          f"max h/h* = {ver['max_ratio']:.2f}")
    # verificación independiente con NetworkX
    ref = nx.dijkstra_path_length(G, origen, destino)
    print(f"Verificación con NetworkX: costo = {ref:.1f} km -> coincide: {abs(ref - ra['costo']) < 1e-6}")

    esc = escalamiento()
    print("\n=== Escalamiento (30 consultas por tamaño) ===")
    print(f"{'nodos':>6}{'Dij exp':>9}{'A* exp':>8}{'Dij ms':>9}{'A* ms':>8}{'reducción':>11}")
    for f in esc:
        print(f"{f['n']:>6}{f['dij_exp']:>9.0f}{f['ast_exp']:>8.0f}{f['dij_ms']:>9.2f}{f['ast_ms']:>8.2f}{f['reduccion']:>10.0f}%")

    noadm = heuristica_no_admisible()
    print("\n=== Heurística no admisible (red de ~600 nodos, 60 consultas) ===")
    for f in noadm:
        print(f"h x{f['escala']:<4} admisible={str(f['admisible']):5}  sobrecosto medio = {f['sobrecosto']:.2f} %  "
              f"rutas no óptimas = {f['no_optimas']}/{f['pares']}  nodos expandidos = {f['expandidos']:.0f}")

    dashboard_regional(G, rd, ra, ver, origen, destino)
    dashboard_escalamiento(esc, noadm)

    # guarda los resultados para reutilizarlos en el informe
    limpio = lambda r: {k: (float(v) if isinstance(v, (np.floating, float)) else v) for k, v in r.items()}
    with open(os.path.join(AQUI, "resultados_s5.json"), "w", encoding="utf-8") as f:
        json.dump({"dijkstra": {k: v for k, v in rd.items() if k != "orden"} | {"orden": rd["orden"]},
                   "astar": {k: v for k, v in ra.items() if k != "orden"} | {"orden": ra["orden"]},
                   "viol_adm": ver["viol_adm"], "viol_con": ver["viol_con"], "max_ratio": float(ver["max_ratio"]),
                   "escalamiento": [limpio(x) for x in esc], "no_admisible": [limpio(x) for x in noadm]},
                  f, ensure_ascii=False, indent=1)


if __name__ == "__main__":
    main()
```

B Anexo B. Salida de la ejecución

```
=== Red regional: 21 ciudades, 33 carreteras ===
           Costo (km)  Expandidos  Tiempo (ms)
Dijkstra         326.9          21        0.086
A*               326.9          12        0.058
Ruta: Atlautla -> Amecameca -> Chalco -> CDMX -> Tepeji -> Tula -> San Juan del Río -> Querétaro
Heurística haversine: violaciones de admisibilidad = 0, de consistencia = 0, max h/h* = 0.80
Verificación con NetworkX: costo = 326.9 km -> coincide: True

=== Escalamiento (30 consultas por tamaño) ===
 nodos  Dij exp  A* exp   Dij ms   A* ms  reducción
   100       52      20     0.10    0.05        61%
   290      136      43     0.31    0.13        68%
   600      325     118     0.77    0.34        64%
  1000      492     144     1.60    0.59        71%
  1500      926     300     3.28    1.42        68%
  2500     1228     404     3.70    1.54        67%

=== Heurística no admisible (red de ~600 nodos, 60 consultas) ===
h x1.0  admisible=True   sobrecosto medio = 0.00 %  rutas no óptimas = 0/60  nodos expandidos = 102
h x1.5  admisible=False  sobrecosto medio = 1.89 %  rutas no óptimas = 45/60  nodos expandidos = 29
h x2.0  admisible=False  sobrecosto medio = 5.38 %  rutas no óptimas = 55/60  nodos expandidos = 19
h x3.0  admisible=False  sobrecosto medio = 7.48 %  rutas no óptimas = 56/60  nodos expandidos = 17
```